MIT | Academy of Engineering

(An Autonomous Institute Affiliated to Savitribai Phule Pune University)

ASSIGNMENT 1.2

Reasoning in Knowledge Based Expert System

MIT Academy of Engineering

(An Autonomous Institute Affiliated to Savitribai Phule Pune University)

Alandi (D), Pune – 412105 (2026–

2027)

Name – Rushikesh Ganesh Shinde.

PRN – 202401040040

Div – A , Batch – A1

Roll No – 9

Course – AIML

COLLEGE TIME TABLE KNOWLEDGE BASE USING PROLOG

TITLE :

College Time Table Knowledge Base Using Prolog

AIM :

To develop a College Time Table Knowledge Base using Prolog that stores information about courses, instructors, classrooms, and timetable schedules, and provides rules to retrieve useful information through logical queries.

EXPERIMENT OBJECTIVE

1. To understand the basic concepts of Prolog programming.
2. To represent college timetable information using facts and rules.
3. To create a knowledge base containing courses, instructors, locations, and schedules.
4. To write Prolog rules for retrieving timetable information.
5. To execute queries for finding instructors, classrooms, courses, and daily schedules.
6. To understand how logical inference and pattern matching work in Prolog.

OUTCOME

1. After completing this experiment, the student will be able to:
2. Create a knowledge base using Prolog facts.
3. Define rules for retrieving information from the knowledge base.
4. Execute Prolog queries to obtain required timetable information.
5. Understand the relationship between facts, rules, variables, and queries.
6. Apply Prolog for solving simple real-world knowledge representation problems.

CONCEPT:

Prolog is a logic programming language commonly used in Artificial Intelligence and knowledge representation. Unlike procedural programming languages, Prolog represents knowledge using facts and rules and answers questions using logical inference.

A Prolog program mainly consists of:

1. Facts

2. Rules

```
% COLLEGE TIME TABLE KNOWLEDGE BASE

% COURSE FACTS

course(dbms).
course(java).
course(ai).
course(computer_graphics).
course(theory_of_computation).
course(computer_networks).
course(software_engineering).
course(mathematics).

% COURSE INSTRUCTOR FACTS
% instructor(Course, Instructor)

instructor(dbms, prof_shinde).
instructor(java, prof_patil).
```

```
instructor(ai, Dr.S.Barve).
instructor(computer_graphics, prof_jadhav).
instructor(theory_of_computation, prof_deshmukh).
instructor(computer_networks, prof_kadam).
instructor(software_engineering, prof_pawar).
instructor(mathematics, prof_more).

% LOCATION FACTS
% location(Course, Room)

location(dbms, lab_301).
location(java, lab_302).
location(ai, lab_303).
location(computer_graphics, lab_304).
location(theory_of_computation, room_201).
location(computer_networks, room_202).
location(software_engineering, room_203).
location(mathematics, room_204).

% TIMETABLE FACTS
% timetable(Day, Time, Course, Instructor, Room)

timetable(monday, '09:00-10:00', dbms, prof_shinde, lab_301).
timetable(monday, '10:00-11:00', java, prof_patil, lab_302).
timetable(monday, '11:00-12:00', ai, prof_kulkarni, lab_303).
```

```
timetable(tuesday, '09:00-10:00', computer_graphics, prof_jadhav, lab_304).
timetable(tuesday, '10:00-11:00', theory_of_computation, prof_deshmukh, room_201).
timetable(tuesday, '11:00-12:00', mathematics, prof_more, room_204).

timetable(wednesday, '09:00-10:00', computer_networks, prof_kadam, room_202).
timetable(wednesday, '10:00-11:00', software_engineering, prof_pawar, room_203).
timetable(wednesday, '11:00-12:00', dbms, prof_shinde, lab_301).

timetable(thursday, '09:00-10:00', java, prof_patil, lab_302).
timetable(thursday, '10:00-11:00', ai, prof_kulkarni, lab_303).
timetable(thursday, '11:00-12:00', computer_graphics, prof_jadhav, lab_304).

timetable(friday, '09:00-10:00', theory_of_computation, prof_deshmukh, room_201).
timetable(friday, '10:00-11:00', computer_networks, prof_kadam, room_202).
timetable(friday, '11:00-12:00', software_engineering, prof_pawar, room_203).

% RULES

% Rule 1:
% Find instructor of a course

course_instructor(Course, Instructor) :-
    instructor(Course, Instructor).
```

```
% Rule 2:
% Find location of a course

course_location(Course, Room) :-
    location(Course, Room).

% Rule 3:
% Find complete timetable information
% for a particular course

course_schedule(Course, Day, Time, Instructor, Room) :-
    timetable(Day, Time, Course, Instructor, Room).

% Rule 4:
% Find all classes conducted by an instructor

teacher_class(Instructor, Course, Day, Time, Room) :-
    timetable(Day, Time, Course, Instructor, Room).

% Rule 5:
% Find classes conducted on a particular day

day_schedule(Day, Course, Time, Instructor, Room) :-
    timetable(Day, Time, Course, Instructor, Room).
```

```
% Rule 6:
% Check whether a course is available

course_available(Course) :-
    course(Course).

% Rule 7:
% Check whether an instructor teaches a particular course

teaches(Instructor, Course) :-
    instructor(Course, Instructor).

% Rule 8:
% Find the room in which a teacher conducts a class

teacher_room(Instructor, Room) :-
    timetable(_, _, _, Instructor, Room).

% Rule 9:
% Find courses conducted in a particular room

room_course(Room, Course) :-
    timetable(_, _, Course, _, Room).

% Rule 10:
% Find courses conducted at a particular time
```

```
class_at_time(Time, Course, Instructor, Room) :-
    timetable(_, Time, Course, Instructor, Room).


%Example
%course_instructor(dbms, X).
%course_location(dbms, X).
%course_schedule(dbms, Day, Time, Instructor, Room).
%day_schedule(monday, Course, Time, Instructor, Room).
%course_available(ai).
%class_at_time('10:00-11:00', Course, Instructor, Room).
```

RESULT:

The College Time Table Knowledge Base was successfully implemented in Prolog. The program stores course, instructor, location, and timetable information as facts and uses logical rules to retrieve required information through queries.

CONCLUSION:

The experiment demonstrates how Prolog can be used for knowledge representation and logical reasoning. By defining facts and rules, different timetable-related queries can be answered automatically. This experiment provides a basic understanding of facts, rules, variables, queries, pattern matching, and logical inference in Prolog.